EBSearch: Topic-Driven Multi-Video Retrieval and Synthesized Reporting over Bilibili

EnhancedBilibiliSearch Project
Technical Report
June 10, 2026

***Abstract*—Answering a research question from video usually means watching several videos and reconciling what they say—slow, and harder still on Bilibili, where the search API must be cryptographically signed, returns no usable relevance score, and mixes summarizable explainers with multi-hour course dumps. We present *EBSearch*, a pipeline that turns a single topic into one cited, synthesized report. It (i) searches Bilibili through the WBI-signed `search/type` endpoint, (ii) ranks and selects the top-N most relevant and *summarizable* videos with a transparent heuristic (and an optional cheap-LLM re-rank), (iii) fans out per-video summarization by *reusing* a deployed AIVideoSummary backend over HTTP—inheriting its cookies, ASR relay, and content-addressed cache—and (iv) synthesizes the per-video summaries into a single report with a stronger model, under a strict provenance contract in which every concrete claim carries a bracketed `[n]` citation that resolves to a source video. A key empirical finding, validated live against the production API, is that Bilibili's `rank_score` field is null in practice, so we rank by *result order* reinforced by hit columns, plays, recency, and a duration-fit window. The system classifies the topic into one of six report types (how-to, news, comparison, concept, review, general) and emits a type-specific schema and rendering. EBSearch shares a unified, stdlib-only account and metering layer with the backend. We give an architectural and cost/latency-reasoning evaluation; all numbers are explicitly illustrative design estimates, not measured benchmarks.**

***Index Terms*—multi-document summarization, information retrieval, large language models, citation-grounded generation, Bilibili, report generation.**

I. Introduction

A common research task is not "summarize *this* video" but "what do the videos on *this topic* collectively say?" Done by hand it means searching, triaging a noisy result list, watching several candidates, and mentally reconciling agreements, disagreements, and gaps. EBSearch automates this loop for Bilibili and emits a single, cited report. The setting imposes concrete constraints that shape the design.

- **Signed, score-less search.** Bilibili's web search endpoint (`wbi/search/type`) requires a per-day WBI request signature, a browser-like `User-Agent`/`Referer` to clear risk-control (HTTP 412), and login cookies for full access. Crucially, the API's `rank_score` field is null in practice, so a system cannot rank by it and must reconstruct relevance from other signals.
- **Noisy candidate pools.** Raw results interleave concise explainers with multi-hour course compilations and off-topic popular clips; the former are ideal to summarize, the latter are expensive or irrelevant.
- **Cost and rate limits.** Each summarized video and each strong-LLM call costs money or tokens, and free-tier providers impose tokens-per-minute caps. The pipeline must be aggressively cost-gated: hard prefiltering, a small configurable cap on how many videos are summarized, subtitle-first reuse, and *exactly one* strong-model synthesis call.

a) Contributions:

- A stdlib-only WBI-signed Bilibili search client and a transparent, order-based ranking heuristic that does *not* use the (null) `rank_score`, with hard prefilters for junk and course-length videos (Sections III-A, III-B).
- A fan-out summarization stage that reuses an existing AIVideoSummary backend over HTTP—fetching subtitles itself to avoid platform risk-control, submitting jobs in parallel, and polling adaptively so wall-clock approximates the slowest single video (Section III-C).
- A citation-grounded synthesis stage: one stronger-model call over a self-contained evidence pack, with a strict `[n]`-provenance contract, deterministic fallbacks, and adaptive report typing into six type-specific schemas (Section III-E).
- Reuse of the unified, stdlib-only account/metering layer shared with the backend, with report-level pre-authorize-then-settle billing (Section III-G).

II. System Architecture

EBSearch is a four-stage pipeline (Table I, Fig. 1) orchestrated by a single `research(topic, cfg)` entry point and exposed as a CLI, a REST API, and a standalone web page. The core is standard-library only; FastAPI/uvicorn are optional server dependencies. Each stage is cost-gated, and exactly one stage makes a strong-model call.

III. Methodology and Implementation

A. WBI-signed search

The search client is standard-library only. It first derives the daily *mixin key* from `/x/web-interface/nav`: the basenames of two returned image URLs are concatenated and reordered by a fixed 64-entry permutation table, and the first 32 characters become the key (cached per client instance for the day). Each request is then signed by appending `wts` (a timestamp) and `w_rid`—the MD5 of the sorted, sanitised, percent-encoded query string concatenated with the mixin key:

TABLE I
EBSEARCH PIPELINE STAGES.

Stage	Output	Responsibility
Search	`VideoHit[]`	WBI-signed multi-plan search; dedup in order
Rank/Select	`ScoredHit[]`	Hard prefilter + heuristic score (+ optional LLM rerank); top-N
Fan-out	`VideoSummary[]`	Reuse AIVideoSummary backend per video (parallel submit, adaptive poll)
Classify	report type	Topic $\rightarrow$ one of six types (heuristic + optional LLM)
Synthesize	`TopicReport`	One strong-model, cited, type-specific report
Render	Markdown	Adaptive presentation by report type

```
 topic
   |
[Search]  WBI-signed search/type  (multi-plan,
   |       duration filter, dedup in result order)
   v
 VideoHit[]  (rank_score is NULL -> use position)
   |
[Rank/Select]  hard prefilter (junk/short/long/
   |           off-topic) -> heuristic score
   |           (position, hit_columns, play,
   |            recency, duration-fit) -> top-N
   |           [+ optional cheap-LLM rerank]
   v
 ScoredHit[N]
   |
[Fan-out]  per video: fetch subtitle (WBI) ->
   |        POST /api/jobs to AIVideoSummary backend
   |        (transcript job; force_asr url job if
   |         only AI captions + ASR enabled);
   |        parallel submit, adaptive poll
   v
 VideoSummary[]  (reuses backend cookies/relay/cache)
   |
[Classify] topic -> {how_to,news,comparison,
   |                 concept,review,general}
   v
[Synthesize] ONE strong-model call over an
   |          evidence pack; strict [n] citations
   v
 TopicReport ---> [Render] adaptive Markdown
```

Fig. 1. Topic-to-report data flow. The fan-out stage reuses the AIVideoSummary backend over HTTP; synthesis is the single strong-model call.

```
mixin_key = "".join(s[i] for i in MIXIN)[:32]
params["wts"] = int(time.time())
q = urlencode(sorted(sanitize(params)))
params["w_rid"] = md5((q + mixin_key)).hexdigest()
```

Requests carry a browser `User-Agent`, the Bilibili `Referer`, and (when configured) cookies parsed from a Netscape file, clearing risk-control. A *search plan* (keyword, order, duration filter, page count) is the unit of work; the query stage can emit several plans (raw, plus optional native "suggest" phrasings and optional LLM query expansion), which the client executes with throttling and merges into a deduplicated list in *first-seen* (approximately relevance) order. The observed caps are 1000 results / 50 pages / 20 per page. Rows with an empty `bvid` or zero duration are dropped at parse time.

B. Ranking without a relevance score

The central empirical finding—validated live, since the canonical community API documentation had been taken down—is that the search API's `rank_score` is null in practice. EBSearch therefore reconstructs relevance from the signals that are actually present. Selection has two layers, both cheap and the second optional.

a) Hard prefilter: A candidate is rejected if it has no `bvid`, zero or sub-threshold duration (drops clips/shorts), a duration above the course cutoff (drops multi-hour compilations that are not cheaply summarizable), near-zero plays, or is *off-topic*—defined as having neither a title/tag hit flagged by Bilibili nor any overlap between the title/tags and the topic's tokens (ASCII words plus CJK runs and bigrams).

b) Transparent weighted heuristic: Survivors receive a weighted score

$$S = w_p\, p + w_h\, h + w_v\, v + w_r\, r + w_d\, d,$$

where the components are: p, a position decay $1/(1{+}0.12{\cdot}\mathrm{pos})$ that trusts Bilibili's result order (the strongest honest signal); h, a hit-column strength favouring title/tag matches over description/author; v, log-damped plays normalised to the pool maximum so one viral course cannot dominate; r, an exponential recency decay (an $\sim$18-month half-life for a fast-moving field); and d, a Gaussian duration-fit bell peaking near a typical explainer length. The default weights are $(w_p, w_h, w_v, w_r, w_d) = (0.34, 0.20, 0.16, 0.12, 0.18)$. Course-length videos that survive a widened window get a multiplicative penalty so they sink below explainers. The reasons behind every score are retained for transparency.

c) Optional cheap-LLM re-rank: When enabled, only the heuristic's top slice is sent to a cheap model for one structured re-rank call; any error falls back safely to the heuristic order. This bounds cost (the full pool is never sent) and never hard-fails.

C. Fan-out summarization by backend reuse

Rather than re-implementing transcription, EBSearch *reuses* a deployed AIVideoSummary backend over HTTP. Because the two services are co-located, the fan-out can hit the backend at `127.0.0.1` and inherit its Bilibili cookies, its overseas ASR relay, and its content-addressed cache—so re-running a topic or summarizing a previously-seen video is free.

a) Subtitle policy: To avoid triggering Bilibili's per-video risk-control through the backend's heavy yt-dlp download, the fan-out *itself* fetches each video's subtitles via light WBI player-API calls and classifies them as `human`, `ai`, or `none`. Human subtitles (best quality, free) are always used by submitting a `transcript` job. When ASR is enabled, videos with only AI captions are re-transcribed by submitting a `force_asr url` job (the backend then skips its subtitle-first path and runs Whisper). When ASR is disabled, AI captions are used as-is rather than nothing. The duration filter and selection cap remain the main cost gates.

b) Parallel submit, adaptive poll: Subtitle fetch and job submission run concurrently across videos through a capped thread pool (the previous serial, one-second-per-video loop was pure latency); the pool is capped to stay gentle on the WBI endpoint. The pipeline then polls outstanding jobs, fast at first (cache hits finish in seconds) and backing off to a steady cadence, until each job is done, errors, or hits a per-video timeout. The net effect is that the fan-out wall-clock is dominated by the *slowest single video* rather than the sum of all of them. Each result records its transcript origin (`subtitle` vs `asr`).

D. Adaptive report typing

Before synthesis the topic is classified into one of six report types (`how_to`, `news`, `comparison`, `concept`, `review`, `general`) by a transport-light classifier: at most one cheap structured LLM call, always backed by a pure-Python keyword heuristic whose rules are ordered by specificity so mixed-signal topics resolve sensibly. The label space is closed; anything not confidently bucketed is `general`. The type drives both the synthesis schema and the rendering.

E. Citation-grounded synthesis

Synthesis makes *exactly one* call to a stronger model (e.g. DeepSeek-v4-pro, or Groq's `gpt-oss-120b` via the overseas relay) and is engineered so that faithfulness is structural rather than hoped-for.

a) Self-contained evidence pack: The model is handed a compact JSON *evidence pack* containing only, per video, a 1-based citation number `n`, its `bvid`, title, author, duration, TL;DR, capped key points and keywords, and chapter titles with timestamps. The prompt forbids any fact not in the pack.

b) Provenance contract: Every concrete claim must carry a bracketed `[n]` marker that resolves to the evidence pack's video `n` (and thus to `sources[n-1]`); the model is told to emit `[n]` only, never raw BV numbers or titles. Timestamps may come only from chapter `start`/`t` values, so the model cannot invent timecodes. The report separates *consensus* from *disagreement* and is required to flag minority/dissenting views explicitly. We chose this structured-sections shape over a comparison-matrix-only or a flowing-brief prompt because it maps 1:1 onto the `TopicReport` object, forces the consensus/dissent split, and makes provenance checkable per claim.

c) Type-specific blocks: The base schema (overview, themes, consensus, disagreements, per-video highlights with timestamps, a ranked watch list, gaps, sources) is augmented by one type-specific block: `steps` for how-to, `timeline` for news, a `comparison` matrix for comparison, `glossary` for concept, and `verdicts` for review. The `general` type reproduces the base prompt verbatim.

d) Cost discipline and graceful degradation: Cost is bounded by sending trimmed evidence (chapter titles only, capped lists) so a report fits within free-tier token-per-minute limits, capping the synthesis output, and keeping `max_videos` small by default. The synthesis caller falls back from the primary provider (e.g. Groq, which can return 413 on its free tier) to a direct DeepSeek call so a report is always produced. If the model returns prose or broken JSON, a fence-aware extractor and deterministic backfill still produce a well-formed `TopicReport` (overview from best-effort text; per-video, sources, and watch list reconstructed straight from the summaries), so the pipeline never hard-fails on a flaky completion.

e) Transport: The LLM client is a tiny stdlib OpenAI-compatible chat function. When a forward proxy is configured it routes the request through an explicit HTTPS `CONNECT` tunnel via `http.client` with `set_tunnel`—*not* `urllib`'s `ProxyHandler`, which does not reliably tunnel HTTPS and can leak the request to the origin IP, re-triggering a provider geo-block (HTTP 403). TLS stays end-to-end to the provider, so the proxy never sees the key or body.

F. Adaptive rendering

The renderer turns a `TopicReport` into Markdown ordered as the reader's journey: overview, themes, consensus vs. disagreements, a sub-topic by video coverage matrix, per-video highlights with clickable `[mm:ss]` deep links (`?t=sec`), a ranked watch list, gaps, and sources. The `[n]` provenance tags are rewritten into Markdown links to the corresponding source video, so every claim remains attributable in the rendered output, and the type-specific block is rendered in a type-appropriate style.

G. Unified account and metering

EBSearch embeds the *same* stdlib-only account subsystem as the backend and points it at the *same* SQLite file and the *same* JWT secret, so a user authenticated against one service is recognised by the other. Storage uses WAL mode with short `IMMEDIATE` transactions and a retry-on-locked loop (two processes, one file); credit changes go through the same atomic check-and-decrement ledger; tokens are the same hand-rolled HS256 JWTs; and phone numbers are salted-SHA-256 hashed for PIPL compliance. Billing is pre-authorize-then-settle at the *report* level: a report's cost is a base plus a per-video charge times the number of videos (plus a premium-model surcharge), pre-authorized when the research job starts and refunded on failure. Because EBSearch already charges its user for the report, its fan-out calls into the backend present a static service key and are *not* metered again there, avoiding double-billing.

IV. Evaluation

We present an architectural and cost/latency-reasoning evaluation. Every number below is an *illustrative design estimate* derived from the architecture, not a measurement, and is labelled as such. **[Insert measured end-to-end timings, token counts, and a screenshot of a rendered report here once a controlled study is run.]**

A. Cost reasoning

Per topic, the dominant costs are (i) the per-video summaries and (ii) the single synthesis call. EBSearch minimises both: the duration filter and hard prefilter cut the pool before any

TABLE II
ILLUSTRATIVE PER-TOPIC COST DECOMPOSITION AT MAX_VIDEOS=4, SUBTITLE-FIRST (*design estimates*, NOT MEASUREMENTS).

Component	Calls	Relative cost
Search (WBI)	several HTTP	negligible (no LLM)
Heuristic rank/select	0 LLM	none
Per-video summary (N)	N cheap	low (caption tier)
Synthesis	1 strong	moderate (the main LLM spend)
Repeat of seen video/topic	cache hit	0 (backend cache)

TABLE III
ILLUSTRATIVE FAN-OUT WALL-CLOCK FOR N VIDEOS (*design estimates*; T_i IS VIDEO i'S SUMMARIZATION TIME).

Scheme	Summarization wall-clock
Serial submit + poll	$\approx \sum_i T_i$
Parallel submit + adaptive poll (EBSearch)	$\approx \max_i T_i$
All cached	$\approx$ seconds (poll cadence)

summary is requested; a small `max_videos` (4 by default) caps the number of summaries; subtitle-first reuse means most summaries are the cheap caption tier; and synthesis is exactly one call over trimmed evidence. Table II sketches the relative spend for a default run.

The optional cheap-LLM re-rank and query expansion add at most one cheap call each and are off by default; the largest *free* quality lever is the duration filter, which removes course/compilation noise at zero cost.

B. Latency reasoning

Because the fan-out submits all jobs in parallel and polls adaptively, the summarization wall-clock approximates the time of the slowest single video, not the sum—and cache hits return almost immediately. Synthesis is a single call. Search and ranking are comparatively cheap. Table III contrasts the serial baseline with the parallel design for N videos.

C. Qualitative quality observations

The structured-sections synthesis with a strict provenance contract was chosen after comparing report shapes on a real fixture; in that exploration the structured form captured consensus, dissent, and gaps faithfully while keeping every claim attributable. The order-based ranking, the off-topic guard, and the course penalty together keep the selected set on-topic and summarizable. **[Insert a rendered example report, the selected-video list with scores, and a faithfulness spot-check here.]**

V. RELATED WORK

EBSearch sits at the intersection of information retrieval, multi-document summarization, and report generation. The per-video stage builds on abstractive video summarization over transcripts (and, transitively, on robust speech recognition of the Whisper family [1] via the reused backend). The topic-level synthesis follows retrieval-augmented generation [2], in which a generator is constrained to retrieved evidence; EBSearch's evidence pack and `[n]` provenance contract are a citation-grounded instance of this idea, aligned with the broader line of work on report generation that requires every claim to carry a verifiable source. Relative to closed-source Bilibili search and summary tools, EBSearch's distinguishing features are its transparent, score-less ranking heuristic; its reuse of an existing summarization backend over HTTP; and its adaptive, type-specific, fully cited report schema.

VI. LIMITATIONS AND COMPLIANCE

a) Geo-relay fragility: Synthesis via an overseas provider depends on a forward proxy that is a single point of failure; the design mitigates this with a direct-DeepSeek fallback, but a relay outage still degrades the strong path.

b) Bilibili ToS and risk-control: Search and subtitle fetching rely on WBI signing, browser-mimicking headers, and login cookies, and are subject to Bilibili's terms of service and evolving risk-control (the canonical community API documentation was taken down, so behaviour must be verified empirically and access throttled conservatively). The null `rank_score` is itself an example of undocumented behaviour that may change.

c) AIGC labelling: The synthesized report is AI-generated and should be labelled as such where required by applicable mainland-China AI-generated-content rules.

d) No real payment yet: The shared credits ledger accounts for usage but there is no integrated payment processor; credits are granted administratively or via invite codes.

e) Single-node, in-memory job store: Research jobs live in an in-process registry on a single worker; a restart clears in-flight jobs. Free-tier provider token-per-minute limits also bound how many videos can be synthesized in one report (the default cap reflects this), and larger reports need a paid tier.

VII. CONCLUSION

EBSearch demonstrates that turning a topic into a trustworthy, cited report over Bilibili is achievable by combining careful platform engineering with disciplined reuse. Signing search requests, declining to trust a null relevance score in favour of result order and a transparent heuristic, reusing an existing summarization backend (and its cookies, ASR relay, and cache) over HTTP, and constraining a single strong-model synthesis call to a self-contained evidence pack with a strict `[n]` provenance contract together yield reports whose every claim is attributable, while keeping cost to essentially one strong call per topic. Adaptive report typing tailors both schema and presentation to the question. Sharing a stdlib-only account and metering layer with the backend turns the pipeline into a meterable service without duplicating infrastructure. Future work includes measured benchmarks to replace the analytical estimates here, a durable job queue, and paid-tier scaling beyond the current per-report video cap.

REFERENCES

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," OpenAI, 2022. https://cdn.openai.com/papers/whisper.pdf

[2] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. https://arxiv.org/abs/2005.11401